

Vérification finale — le brief, point par point, contre la preuve

Première version le 23/09/2026, mise à jour le 24/09 (v2.7.0), puis **le 25/09/2026 après v2.7.1** : traçabilité complète d'une analyse, traces Jaeger, et les retours de la séance du 25/09 appliqués à Mardik. Chaque ligne a été **vérifiée** par une commande le 25/09, pas de mémoire. Dépôt `Ganda15/mardik-api-mlops_v2`, `main` = `e22d098`, version en service **v2.7.1**.

Sommaire

1. Les 10 tests d'acceptance fournis	2
2. Au-delà des tests : prouvé en vrai	2
3. Traçabilité d'une analyse — et ce que montre Jaeger	3
L'arbre des spans (parents et enfants)	3
Une trace réelle dans Jaeger	4
4. Les 5 livrables	4
5. Retours de la séance du 25/09, appliqués à Mardik	4
6. Les questions de réflexion du brief	5
7. Ce qui n'est pas fait — dit tel quel	6
8. Liens — vérifiés le 25/09 (réponse 200)	6

1. Les 10 tests d'acceptance fournis

Commande : `pytest tests/acceptance -v` → **10 passed** (25/09).

#	Test du brief	Test fourni	Résultat
1	document long → /v2 selon le contrat, avec score	<code>test_contrat_v2_long_analyse_sans_troncature</code>	✓ PASSED
2	requête invalide → erreur explicite	<code>test_erreurs_explicites_jamais_de_500</code>	✓ PASSED
3	client v1 inchangé pendant le canary	<code>test_client_v1_fonctionne</code>	✓ PASSED
4	fusion → version étiquetée, version identifiable	<code>test_etiquetage_version_apres_gate</code>	✓ PASSED
5	gate en échec → livraison bloquée	<code>test_gate_evaluation_note_par_version</code>	✓ PASSED
6	dérive → rollback via la chaîne, tracé	<code>test_rollback_en_une_operation</code>	✓ PASSED
7	métriques conformes → promotion ; sinon non	<code>test_promotion_canary_puis_totale</code>	✓ PASSED
8	cas à faible confiance → jeu d'éval → rejoué	<code>test_evaluation_enrichie_latence_et_cout</code>	✓ PASSED
9	tableau de bord : 5 signaux visibles	<code>test_dashboard_par_version</code>	✓ PASSED
10	journal : tout ajustement avec son signal	<code>test_journal_derive_alerte_jamais_de_rollback_au_tomatique</code>	✓ PASSED

Suite complète : **251 passed** · couverture **85 %** (seuil 80 % appliqué par la chaîne) · `ruff` : aucune erreur · test des données : 12 contrats + 1 cas de production valides.

2. Au-delà des tests : prouvé en vrai

Exigence	Preuve réelle	Fichier
Fusion → version étiquetée, publiée par la chaîne	16 PR fusionnées, 14 versions <code>v2.0.0</code> → <code>v2.7.1</code> ; run <code>main</code> après PR #16 : 7/7	GitHub Actions
Gate réel sur le vrai modèle	18 passages : 15 passés, 3 refusés (latence) ; dernier (PR #16) : rappel 1,000 , précision 0,974 , P95 6 858 ms , coût moyen 0,0326 €	<code>gate-reel.yml</code> , run <code>36022289593</code>
Gate en échec → bloqué	les 3 passages refusés n'ont pas reçu de tag <code>eval-ok</code> ; la publication de PR #12 a été refusée faute de tag	<code>preuves/...gate-reel-ECHEC-latence...txt</code>
Vrai modèle identifié dans la version	manifeste : modèle, commit, empreinte, note du gate, signature	<code>ops/registry/<version>/manifest.json</code>
Rollback sur signal, décision humaine	alerte médiane 0,542 → refus de promotion → clic nommé → <code>v1.0.0</code>	<code>preuves/...derive-reelle-*</code>
Traçabilité d'une analyse (v2.7.1)	<code>request_id</code> dans la réponse, les métriques, la trace et les journaux ; traces et journaux conservés sur disque ; commande <code>ops.tracer</code>	§3 ci-dessous ; <code>tests/test_tracabilite.py</code> (15 tests)

Exigence	Preuve réelle	Fichier
Score de confiance calibré (v2.6.0, v2.7.0)	Platt appris sur 431 clauses réelles ; ECE par contrat 0,026 → 0,0004 ; plafond 0,993	models/v2/config.yaml
Latence < 8 s	c12 : 2,75 – 4,20 s ; gate réel P95 4,4 – 7,6 s sur les passages réussis, 6,9 s au dernier	preuves/...latence-ou-passent...txt
Coût < 0,15 €	0,033 € moyen, tarifs catalogue entrée/sortie	preuves/...cout-reel...txt
v1 intacte	/v1 : 200, exactement clauses, modele, tronque, version	vérifié le 23/09

3. Traçabilité d'une analyse — et ce que montre Jaeger

Question → Réponse courte. *Peut-on retrouver l'historique complet d'une analyse ?* Oui : chaque requête reçoit un `request_id`, présent dans la réponse (succès comme erreur), la ligne de métriques, la trace et les journaux. `python -m ops.tracer <request_id>` réunit les six rubriques.

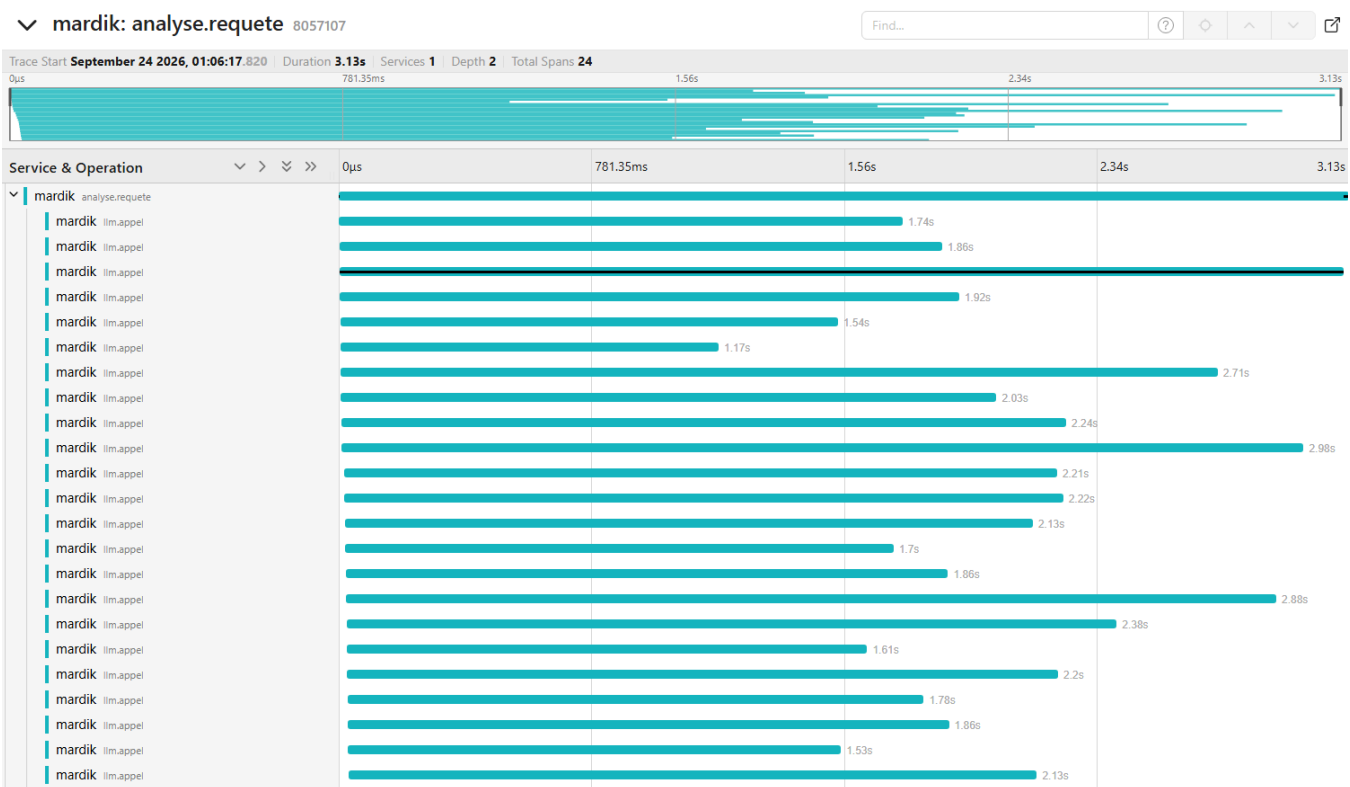
La définition demande...	Où Mardik le garde
quelle version a répondu	en-tête <code>X-Mardik-Version</code> (erreurs comprises) ; champ <code>version</code>
quel modèle a été appelé	champ <code>modele</code> ; attribut de trace <code>llm.modele</code> (v2.7.1) ; manifeste = modèle visé
combien de temps	<code>latence_ms</code> ; durée de chaque span dans la trace
quels seuils ont été évalués	libellé de certitude ; seuils lus au commit de la version ; gate <code>eval-ok/<sha></code> ; watcher
quelle décision a été prise	<code>ops/registry/journal.jsonl</code> : signal, valeur, seuil, acteur, état avant/après

L'arbre des spans (parents et enfants)

```
analyse.requete (parent, app/api_v2.py:110)
  attributs : mardik.request_id · mardik.version · llm.modele · mardik.sections
    └─ llm.appel (enfant, app/api_v2.py:123) – un par section, lancés EN PARALLÈLE (parallelisme: 32)
      └─ attributs : mardik.section · llm.latence_ms · llm.tokens
        └─ llm.appel ...
          └─ llm.appel ...

exportés vers : Jaeger (OTLP, port 4318) ET ops/traces.jsonl (conservé sur disque) –
app/telemetry.py:173-190
```

Une trace réelle dans Jaeger



Analyse réelle du **24/09 à 01:06** (version servie `v2.0.0`), `request_id` `req_470047a4c63f` : **3,13 s** au total, **23 sections** donc 23 appels au modèle, partis **en même temps**. Le plus rapide : 1,17 s ; le plus lent : **3,11 s (section 2)** — c'est lui qui fixe la durée totale : l'analyse attend le plus lent. Sans la trace, on sait seulement que l'analyse a duré 3 s ; avec elle, on sait **où** optimiser.

Cette trace est **antérieure à v2.7.1** : elle n'a pas encore l'attribut `llm.modele`, le modèle appelé n'y est donc pas enregistré — c'est précisément le manque que v2.7.1 ferme.

4. Les 5 livrables

Livable	Où	État
Dossier de conception	docs/livrables/ (4 PDF, 4 schémas <code>.drawio</code> + <code>.png</code>)	✅ envoyé le 18/09, validé
API v2, v1 intacte	app/api_v2.py, app/api_v1.py (fourni, non modifié)	✅
Chaîne LLMops (yaml)	.github/workflows/llmops.yaml, gate-reel.yaml, rollback.yaml	✅
Observabilité : tableau de bord + 3 boucles + journal + traces	ops/dashboard.py, ops/watcher.py, app/pilotage.py, ops/registry/journal.jsonl, ops/tracer.py	✅
Client frontend par un lien	app/static/index.html, scripts/demo.py public	✅ (lien éphémère)

5. Retours de la séance du 25/09, appliqués à Mardik

Les remarques de la séance portaient sur une autre présentation ; chacune est ici **vérifiée contre le code de Mardik**.

Remarque	État dans Mardik (vérifié le 25/09)	Suite
« Tout le travail de télémétrie pour une ligne » : être précis sur ce qui est tracé	2 types de span : analyse.requete et un llm.appe1 par section (23 sur la trace montrée). Pas de span pour le découpage (api_v2.py:114-117), la consolidation (161), la notation (162), la preuve des extraits (165), la calibration (198), ni pour la décision de routage de la gateway (canary v1/v2). Les métriques et journaux couvrent l'analyse entière	Manque reconnu : un span enfant par étape + un span gateway.routage avec la version choisie
Le schéma doit montrer les spans parents et enfants	l'arbre est ajouté au §3, avec une trace réelle	✅ fait dans ce document
Où sont les traces et les métriques dans le code ?	spans : app/api_v2.py:110-127 ; export : app/telemetry.py:173-190 ; métriques : api_v2.py:138 (échec) et :171 (succès) → ops/metrics.json1	✅
Pourquoi un score de confiance à 0 ?	deux causes, dans le code : extrait introuvable mot pour mot dans le texte → clause à 0 (app/pipeline/confiance.py:43) ; aucune clause prouvée (ex. texte qui n'est pas un contrat) → score global 0 (confiance.py:45), libellé « basse »	✅ expliqué, jamais un chiffre rassurant
Les branches de correctif doivent déclencher les évals	le gate se déclenche sur toute PR , quelle que soit la branche (pull_request sans filtre de branche), si un fichier sensible change ; et toute publication exige le tag eval-ok/<sha> du vrai gate. Manque trouvé : le filtre de chemins ne liste pas eval/*.py (code des évals), app/llm_client.py (appel au modèle) ni app/api_v2.py (score global, calibration)	À corriger : ajouter ces chemins au filtre (llmops.yml , filtre-chemins)
Mesurer la durée des étapes de la CI	run main du 24/09 : lint 8 s · tests 26 s (dont couverture 8 s) · gate simulé 16 s · build 44 s (dont envoi de l'image 38 s) · publication 19 s · canary 16 s ; gate réel : 48 s d'appels Azure	l'étape la plus longue est l'image Docker , pas les tests : piste = cache des couches Docker
Montrer l'historique des branches	16 PR fusionnées, toutes par PR, main protégée (checks exigés, admins inclus)	✅ git log --first-parent main
Étape de « rejeu de l'évaluation » sur main = erreur	sur main , le job d'éval tourne en MOCK (16 s, gratuit) : il ne prouve rien sur le vrai modèle — la preuve est le tag eval-ok/<sha> . Il reste pour le rejeu des cas de production (test 8 du brief) et le calcul de version	à alléger : garder le rejeu des cas de production, retirer l'éval MOCK redondante
A/B : la métrique se définit avant le test	seuils du canary écrits avant la promotion : eval/thresholds.yml (erreurs ≤ stable + 2 pts, scores bas ≤ + 5 pts, médiane ≥ stable – 0,05, 30 requêtes minimum)	✅
LangFuse ou Sentry pour les traces	Jaeger (OTLP) + fichier ops/traces.json1 . Sentry écarté : non exigé par le brief, et il envoie par défaut le corps JSON des requêtes (le texte du contrat) — RGPD	décision argumentée

6. Les questions de réflexion du brief

Toutes les questions ont une réponse écrite, une ligne chacune : **annexe A** de Mardik-Conception-Chantier1.md (23 questions) et de Mardik-Conception-Chantier2.md (13 questions).

Réponse du 18/09	Construit	Tracé dans
« aucune découpe : un appel unique, plafond 413 »	map-reduce par sections (imposé par un test fourni) ; 413 en v2.5.0	docs/spec-v2-perimetre.md §6, app/api_v2.py

Réponse du 18/09	Construit	Tracé dans
route <code>/v2/analyses</code> , score « calibré »	<code>/v2/analyse</code> (dépôt) ; score calibré v2.6.0 et v2.7.0	<code>models/v2/config.yaml</code>
routeur lisant <code>weights.json</code>	index du registre (<code>canary_percent</code>)	<code>docs/spec-ch2-pilotage.md</code> §6

7. Ce qui n'est pas fait — dit tel quel

- **Spans manquants** : découpage, consolidation, notation, preuve, calibration et routage de la gateway ne sont pas des spans (§5). La durée de ces étapes n'est donc pas visible dans Jaeger.
- **Filtre d'évals incomplet sur les PR** : `eval/*.py` , `app/llm_client.py` et `app/api_v2.py` n'y sont pas. Aucune version ne peut pourtant être publiée sans le tag `eval-ok` du vrai gate.
- Jaeger garde les traces **en mémoire** : un redémarrage les efface ; la copie durable est `ops/traces.jsonl` .
- Le tarif Azure de la ressource n'est pas confirmé sur le portail : 0,033 € est un coût **au catalogue**.
- La latence du fournisseur dépasse parfois 8 s : **3 passages sur 18** refusés par le gate.
- Les 13 erreurs du modèle sont toutes « garantie » : une définition plus précise dans le prompt reste à faire.
- L'adresse du lien public change à chaque redémarrage du tunnel.

8. Liens — vérifiés le 25/09 (réponse 200)

- Code à la version livrée (lien de tag, ne bouge plus) : https://github.com/Ganda15/mardik-api-mlops_v2/tree/v2.7.1
- Les livrables, lisibles dans le navigateur : https://ganda15.github.io/mardik-api-mlops_v2/
- La traçabilité (PR #16) : https://github.com/Ganda15/mardik-api-mlops_v2/pull/16
- Le dernier gate réel : https://github.com/Ganda15/mardik-api-mlops_v2/actions/runs/36022289593